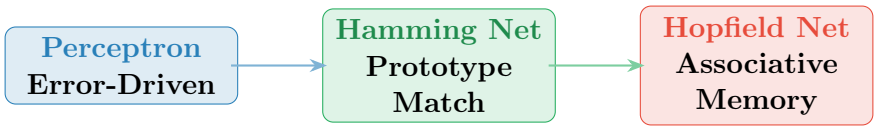
Neural Pattern Classification in C

A Comparative Study of Perceptron, Hamming Network, and Hopfield Network for Bipolar Fruit Pattern Recognition



Author: Md. Kais
Department: Computer Science & Engineering
University: University of Chittagong
Course: CSE 816 — Machine Learning Lab
Repository: https://github.com/Md-Kais/Classification_Algorithms_In_Machine_Learni
Source File: `neural_net_classifier.c` (704 lines, C99)
Date: April 13, 2026

Contents

1. Neural Networks and the Roots of Pattern Recognition

Neural networks trace their conceptual origin to McCulloch and Pitts' 1943 model of the biological neuron [1]. The dream behind that work was simple yet profound: if a single nerve cell fires or stays silent based on the weighted sum of its inputs, then a collection of such cells should be able to learn almost any input-output mapping. Decades of research transformed this idea into a rigorous mathematical discipline.

This report investigates three foundational architectures that emerged from that tradition:

- **Rosenblatt's Perceptron** (1957) — the first learning algorithm proven to converge on linearly separable data [2].
- **The Hamming Network** (Lippmann 1987) — a two-layer competitive network that selects the class whose prototype is closest in Hamming distance to the input [4].
- **Hopfield's Associative Memory** (1982) — a fully connected recurrent network that stores patterns as stable energy minima and recalls them from partial or noisy inputs [3].

All three are implemented in C99 in a single source file, `neural_net_classifier.c`, operating on a bipolar-encoded fruit dataset and reporting classification metrics via the shared `ml_common` library.

2. The Fruit Recognition Challenge: Classifying Bipolar Patterns

2.1. Task Definition

Given a fruit described by three bipolar attributes, classify it as either ORANGE or APPLE.

Feature	**Meaning**	$+1$	-1
`shape`	Geometric profile	Round	Elongated
`texture`	Surface feel	Smooth	Rough
`weight`	Relative mass	Heavy	Light

Both the features and the class targets are encoded in the **bipolar** (or *signed binary*) representation $\{-1, +1\}$, which is mathematically advantageous for both Hebbian weight construction and the bipolar step activation function.

2.2. Why Bipolar Encoding?

In binary encoding, a "0" carries no information in a product $w \cdot 0 = 0$. In bipolar encoding, -1 actively reinforces the opposite direction: $w \cdot (-1) = -w$. This double-sided signal is critical for the Hopfield network's outer-product weight rule and for the Perceptron's error signal to correct in both directions.

3. Background Theory

3.1. Prototype Patterns and Bipolar Class Representations

A **prototype** is a single representative vector for each class, derived from training data. In this implementation, the prototype for each class is computed from the training split by computing the per-feature mean and then binarising:

$$p_c[f] = \begin{cases} +1 & \text{if } \dfrac{1}{|T_c|}\sum_{i \in T_c} x_i[f] \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{1}$$

where T_c is the set of training indices for class c.

This yields one bipolar prototype vector per class. All three networks in this programme consume these prototypes as their primary input to the learning phase (Hamming and Hopfield) or use the training samples directly (Perceptron).

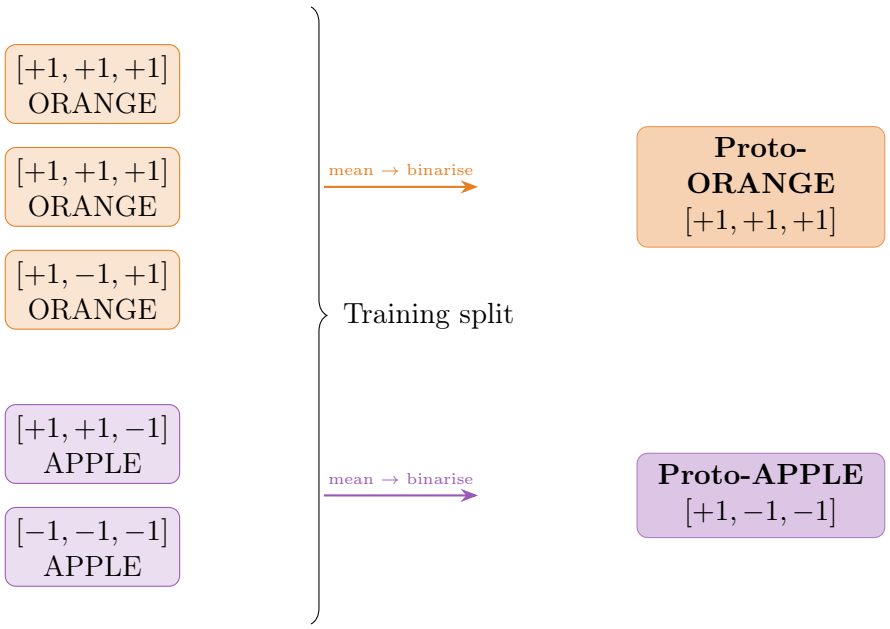


Figure 1: Prototype extraction: per-class feature means are binarised to ± 1 to form the representative prototype vector used by the Hamming and Hopfield networks.

3.2. The Perceptron: A Threshold Logic Unit That Learns

The Perceptron [2] is a single linear threshold unit:

$$\text{net} = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \text{bipolarStep}(\text{net}) = \begin{cases} +1 & \text{net} \geq 0 \\ -1 & \text{net} < 0 \end{cases} \tag{2}$$

where $\mathbf{w} \in \mathbb{R}^3$ is the weight vector and $b \in \mathbb{R}$ is the bias, both initialised to zero.

Internally, the programme maps the integer label to a bipolar target:

$$y_{\text{bipolar}} = \begin{cases} +1 & \text{label} = \text{ORANGE} \\ -1 & \text{label} = \text{APPLE} \end{cases} \tag{3}$$

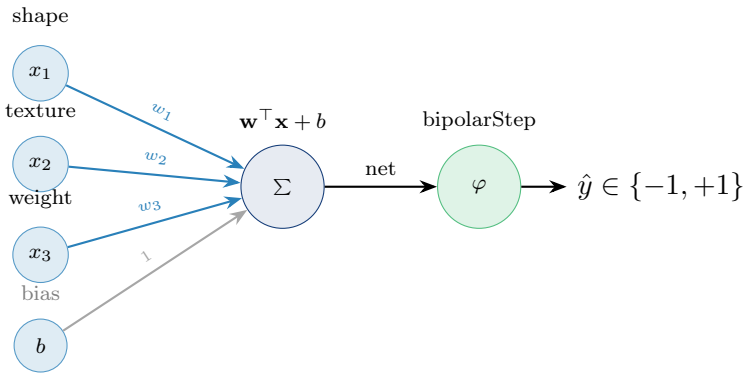


Figure 2: Perceptron architecture for the fruit classification task. Three bipolar input features feed into a linear combiner; a bipolar step function produces the final label $\in \{-1, +1\}$.

3.3. The Perceptron Learning Rule: Error-Corrective Weight Updates

The weight update rule fires only when the predicted and true bipolar labels disagree:

$$\text{if } \hat{y}_{\text{bipolar}} \neq y_{\text{bipolar}} : \begin{cases} \mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot y_{\text{bipolar}} \cdot \mathbf{x} \\ b \leftarrow b + \alpha \cdot y_{\text{bipolar}} \end{cases} \tag{4}$$

where $\alpha = 0.10$ is the learning rate fixed in the source code. Training continues for at most 250 epochs. An epoch that produces zero weight updates terminates training early. The best model (highest validation accuracy) is checkpointed and returned.

3.4. The Hamming Network: Distance-Minimising Prototype Competition

The Hamming Network [4] is a two-layer architecture. The first (feedforward) layer computes the inner product of the input with each stored prototype; the second (MAXNET) layer runs a winner-take-all competition.

3.4.1. Feedforward Layer

$$a_c = \underbrace{\frac{n}{2}}_{\text{bias}} + \sum_{f=1}^{n} W_{cf} \cdot x_f, \qquad W_{cf} = \frac{p_c[f]}{2} \tag{5}$$

where $n = 3$ is the number of features and p_c is the prototype for class c. This activation a_c equals the number of positions where $\mathbf{x}$ agrees with $\mathbf{p}_c$ (i.e. the Hamming similarity).

3.4.2. MAXNET Competitive Layer

The MAXNET iteratively suppresses all units except the winner:

$$a_c^{(t+1)} = \max\Big(0,\ a_c^{(t)} - \varepsilon \sum_{k \neq c} a_k^{(t)}\Big), \quad \varepsilon = \frac{1}{C+1} \tag{6}$$

where $C = 2$ is the number of classes. Iteration stops when at most one unit remains active. The winning class ($\arg\max_c a_c$) is the prediction.

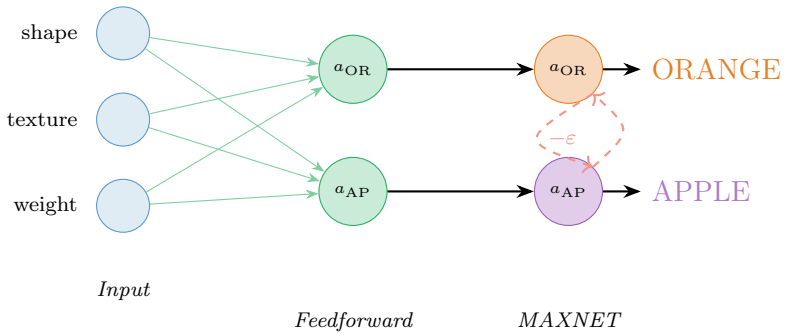


Figure 3: Hamming Network: the feedforward layer computes Hamming similarities; the MAXNET layer iteratively inhibits lower activations until only the winner survives.

3.5. The Hopfield Network: Patterns as Energy Minima

The Hopfield Network [3] is a single-layer recurrent network. Patterns are stored by constructing a weight matrix via the **Hebbian outer-product rule** and recalled by iterating a synchronous update rule until convergence to a stable fixed point.

3.5.1. Weight Construction (Hebbian Rule)

$$W_{ij} = \frac{1}{n} \sum_{k=1}^{P} p_k[i]\, p_k[j], \quad i \neq j; \qquad W_{ii} = 0 \tag{7}$$

where $P = 2$ is the number of stored patterns (one per class) and $n = 3$ is the feature dimension. The zero diagonal enforces that a neuron does not excite itself.

3.5.2. Synchronous Recall Rule

Starting from the input $\mathbf{x}$ as the initial state:

$$s_i^{(t+1)} = \text{bipolarStep}\left(\sum_{j \neq i} W_{ij}\, s_j^{(t)} \right) \tag{8}$$

Iteration stops when no unit changes its state or the maximum iteration count (25) is reached. If the final state matches a stored prototype exactly, the corresponding label is returned. Otherwise, the recall is classified as **spurious** and the nearest prototype is used.

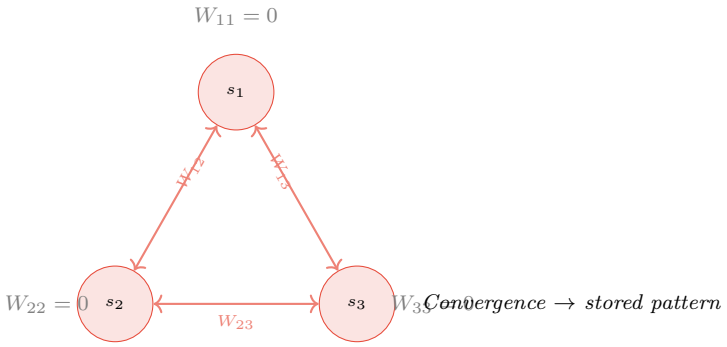


Figure 4: Hopfield Network for three bipolar units ($n = 3$). All non-diagonal pairs are bidirectionally connected with symmetric weights $W_{ij} = W_{ji}$. Self-connections are always zero. The network converges to the nearest stored energy minimum.

4. Research Objectives

The overarching aim of this laboratory exercise is to implement, evaluate, and compare three neural network paradigms in standard C99 on a common bipolar pattern-classification task. The specific objectives are:

O1. Implement a Perceptron classifier with bipolar activations, error-corrective learning, and best-validation checkpointing.

O2. Implement a Hamming Network with feedforward prototype-matching and MAXNET competition.

O3. Implement a Hopfield Associative Memory with Hebbian weight construction, synchronous recall, and spurious-state detection.

O4. Apply a shared evaluation framework (`ml_common.h`) to compare all three networks on identical train / validate / test splits of the fruit dataset.

O5. Provide a complete, reproducible C implementation usable without any external library beyond the standard C math library.

5. Formal Algorithms

5.1. Network Training Procedures

Algorithm 1 Perceptron Training with Validation Checkpointing

Require: Training split $\mathcal{T}$, validation split $\mathcal{V}$, learning rate $\alpha = 0.10$, max epochs $E = 250$

$\mathbf{w} \leftarrow [0, 0, 0]$, $b \leftarrow 0$
$\mathbf{w}_{\text{best}} \leftarrow \mathbf{w}$, $b_{\text{best}} \leftarrow b$, $\text{acc}_{\text{best}} \leftarrow -\infty$
for epoch $= 1$ **to** E **do**
    updates $\leftarrow 0$
    for each $(\mathbf{x}_i, y_i) \in \mathcal{T}$ **do**
        $\hat{y} \leftarrow \text{bipolarStep}(\mathbf{w}^\top \mathbf{x}_i + b)$
        if $\hat{y} \neq y_i^{\text{bip}}$ **then**
            $\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot y_i^{\text{bip}} \cdot \mathbf{x}_i$
            $b \leftarrow b + \alpha \cdot y_i^{\text{bip}}$
            updates += 1
        end if
    end for
    acc $\leftarrow \text{accuracy}(\mathbf{w}, b, \mathcal{V})$
    if acc $> \text{acc}_{\text{best}} + \epsilon$ **then**
        $(\mathbf{w}_{\text{best}}, b_{\text{best}}) \leftarrow (\mathbf{w}, b)$; $\text{acc}_{\text{best}} \leftarrow$ acc
    end if
    if updates $= 0$ **then break**
    end if ▷ Perfect training classification
end for
return $(\mathbf{w}_{\text{best}}, b_{\text{best}})$

Algorithm 2 Hamming Network Initialisation (Training)

Require: Prototypes $\mathbf{p}_{\text{OR}}$, $\mathbf{p}_{\text{AP}} \in \{-1, +1\}^3$, number of features $n = 3$, number of classes $C = 2$

for $f = 0$ **to** $n - 1$ **do**
    $W_{0,f} \leftarrow p_{\text{OR}}[f]/2$ ▷ Orange row
    $W_{1,f} \leftarrow p_{\text{AP}}[f]/2$ ▷ Apple row
end for
$\text{bias}[0] \leftarrow \text{bias}[1] \leftarrow n/2 = 1.5$
$\varepsilon \leftarrow 1/(C + 1) = 1/3$
return $(W, \text{bias}, \varepsilon)$

Algorithm 3 Hopfield Network Weight Construction (Hebbian Training)

Require: Prototypes $\{\mathbf{p}_k\}_{k=0}^{P-1}$, $P = 2$, $n = 3$

$W \leftarrow \mathbf{0}_{n \times n}$
for $k = 0$ **to** $P - 1$ **do**
  for $i = 0$ **to** $n - 1$ **do**
    for $j = 0$ **to** $n - 1$ **do**
      if $i \neq j$ **then**
        W_{ij} += $p_k[i] \cdot p_k[j]$
      end if
    end for
  end for
end for
for $i = 0$ **to** $n - 1$ **do**
  for $j = 0$ **to** $n - 1$ **do**
    $W_{ij} \leftarrow W_{ij}/n$ ▷ Normalise by dimension
  end for
end for
return W

5.2. Network Inference Procedures

Algorithm 4 Hamming Network Prediction (MAXNET Recall)

Require: Input $\mathbf{x}$, weights W, bias, ε, max MAXNET iterations = 30

for $c = 0$ **to** $C - 1$ **do**
  $a_c \leftarrow \text{bias}[c] + \sum_f W_{cf} \cdot x_f$
end for
for iter $= 1$ **to** 30 **do**
  for $c = 0$ **to** $C - 1$ **do**
    $a'_c \leftarrow \max\big(0,\ a_c - \varepsilon \sum_{k \neq c} a_k\big)$
  end for
  $a \leftarrow a'$
  if at most one $a_c > 0$ **then break**
  end if
end for
return $\arg\max_c a_c$

Algorithm 5 Hopfield Network Prediction (Synchronous Recall)

Require: Input $\mathbf{x}$, weight matrix W, prototypes $\{\mathbf{p}_k\}$, max iterations $= 25$

$\mathbf{s} \leftarrow \mathbf{x}$ ▷ Initialise state from input
for iter $= 1$ **to** 25 **do**
  $\mathbf{s}' \leftarrow \mathbf{s}$, changed $\leftarrow 0$
  for $i = 0$ **to** $n - 1$ **do**
    $\text{net}_i \leftarrow \sum_j W_{ij} \cdot s_j$
    $s'_i \leftarrow \text{bipolarStep}(\text{net}_i)$
    if $s'_i \neq s_i$ **then** changed $+= 1$
    end if
  end for
  $\mathbf{s} \leftarrow \mathbf{s}'$
  if changed $= 0$ **then break**
  end if
end for
for $k = 0$ **to** $P - 1$ **do**
  if $\mathbf{s} = \mathbf{p}_k$ **then return** class k, spurious $= 0$
  end if
end for
return nearestPrototype($\mathbf{s}$), spurious $= 1$

6. Dataset Description and File Format

6.1. The Fruit CSV Dataset

The programme reads `Data/fruit_dataset.csv`. Each row encodes one labelled fruit sample.

6.1.1. Column Layout

Column	Name	Allowed Values
1	`shape`	$+1$ (round) or -1 (elongated)
2	`texture`	$+1$ (smooth) or -1 (rough)
3	`weight`	$+1$ (heavy) or -1 (light)
4	`label`	`ORANGE` or `APPLE`

The label column also accepts `orange`/`apple` (lowercase), `+1`/`1` (ORANGE), and `-1`/`0` (APPLE) for robustness. An optional header row is auto-detected: if the first field of line 1 is non-numeric, it is skipped.

6.1.2. Sample File Content

```
shape,texture,weight,label
+1,+1,+1,ORANGE
+1,+1,+1,ORANGE
+1,-1,+1,ORANGE
+1,+1,-1,APPLE
-1,-1,-1,APPLE
+1,-1,-1,APPLE
```

```
+1,+1,+1,ORANGE
-1,+1,-1,APPLE
```

Listing 1: Data/fruit_dataset.csv (excerpt)

6.2. Deterministic Train / Validate / Test Split Strategy

The programme does not use separate train and test files. Instead, it loads the single CSV and partitions it **deterministically** using a fixed random seed (`FRUIT_SPLIT_SEED = 815`):

Split	Ratio	Purpose
Train	60%	Model fitting and prototype computation
Validate	20%	Best-epoch selection (Perceptron)
Test	20%	Final held-out evaluation

The shuffle uses `srand(815)` followed by a Fisher-Yates pass over the index array, guaranteeing that the same dataset always produces the same partition.

7. The C Programme: `neural_net_classifier.c`

The complete source is available at:

https://github.com/Md-Kais/Classification_Algorithms_In_Machine_Learning/blob/main/neural_net_classifier.c

The file is 704 lines of C99. It is self-contained apart from `ml_common.h` / `ml_common.c` which supply the shared evaluation layer. The high-level structure is shown in Figure 5.

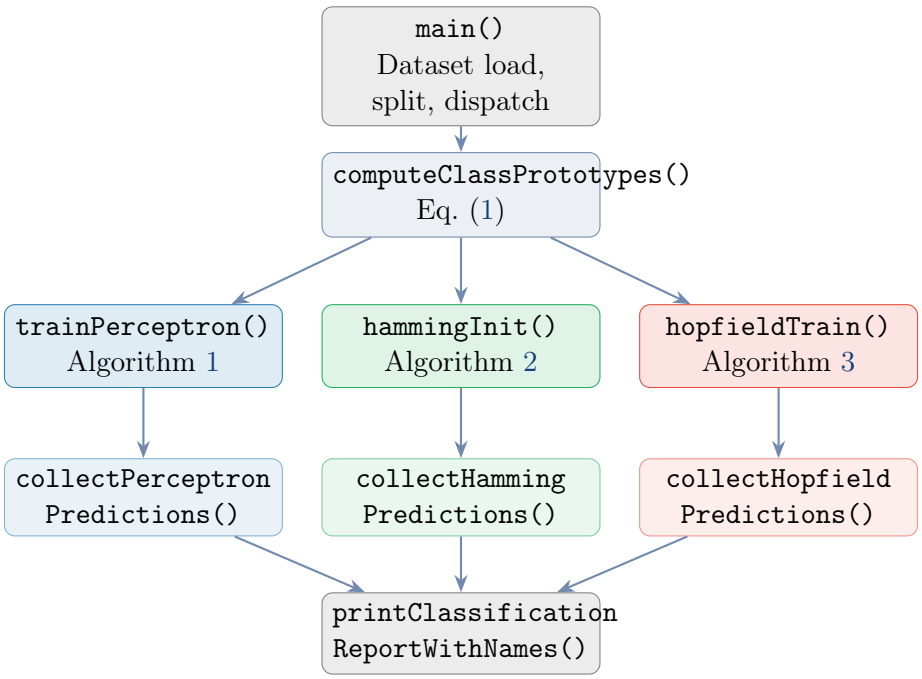


Figure 5: Call graph of `neural_net_classifier.c`. All three networks share the same prototype computation and evaluation reporting layer.

8. Anatomy of the C Implementation

8.1. Header Files and Compile-Time Constants

```c
#include "ml_common.h"
#include <math.h>
#include <stdlib.h>
#include <string.h>

#define FRUIT_NUM_FEATURES  3
#define FRUIT_LABEL_ORANGE  0
#define FRUIT_LABEL_APPLE   1
#define FRUIT_TRAIN_RATIO   0.60
#define FRUIT_VALID_RATIO   0.20
#define FRUIT_SPLIT_SEED    815

#define PERCEPTRON_LR       0.10
#define PERCEPTRON_MAX_EPOCHS 250
#define HOPFIELD_MAX_ITER   25
#define MAXNET_ITER         30
```

Listing 2: Includes and macro definitions at the top of `neural_net_classifier.c`

`ml_common.h` provides `NUM_CLASSES` (=2), `safeDivide()`, `labelToStrCustom()`, `buildClassificationReportFromArrays()`, and `printClassificationReportWithNames()`, among other utilities. All constant definitions that govern learning rate, epoch count, and iteration limits are centralised in the macro block above, making them trivial to adjust.

8.2. Bipolar Activation Function and Label Conversion

```c
static int toBipolarLabel(int label)
{
    return (label == FRUIT_LABEL_ORANGE) ? +1 : -1;
}

static int bipolarStep(double net)
{
    return (net >= 0.0) ? +1 : -1;
}
```

Listing 3: The two core utility functions used throughout the file

`bipolarStep` implements the hard-limiter activation function (Equation (2)). The threshold is at zero: net $\geq 0 \Rightarrow +1$. `toBipolarLabel` converts the integer class label into the bipolar target required by the Perceptron update rule (Equation (3)).

8.3. Dataset Loading: Parsing the Fruit CSV

```c
while (fgets(line, sizeof(line), fp) != NULL) {
    line_no++;
    if (line[0] == '\n' || line[0] == '\r' || line[0] == '\0')
    continue;

    char *shape_tok   = strtok(line,  ",");
    char *texture_tok = strtok(NULL, ",");
    char *weight_tok  = strtok(NULL, ",");
```

```
    char *label_tok   = strtok(NULL, ",\r\n");

    if (!shape_tok || !texture_tok || !weight_tok || !label_tok) {
        fprintf(stderr, "FATAL: invalid fruit CSV format ...\n");
        exit(EXIT_FAILURE);
    }
    /* Auto-detect and skip a non-numeric header row */
    if (line_no == 1) {
        char *endptr = NULL;
        strtod(shape_tok, &endptr);
        if (endptr == shape_tok || ...) continue;
    }
    ds.data[ds.size].x[0] = atof(shape_tok);
    ds.data[ds.size].x[1] = atof(texture_tok);
    ds.data[ds.size].x[2] = atof(weight_tok);
    ds.data[ds.size].label = parseFruitLabel(label_tok);
    ds.size++;
}
```

Listing 4: Core CSV parsing loop inside `loadFruitDataset()`

The parser handles dynamic memory growth (doubles capacity on overflow with `realloc`), automatic header skipping, and multi-label string matching in `parseFruitLabel`.

8.4. Prototype Computation from the Training Split

```
static void computeClassPrototypes(const FruitDataset *ds,
                                   const FruitSplit   *train,
                                   double proto_orange[
    FRUIT_NUM_FEATURES],
                                   double proto_apple [
    FRUIT_NUM_FEATURES])
{
    double orange_sum[FRUIT_NUM_FEATURES] = {0.0, 0.0, 0.0};
    double apple_sum [FRUIT_NUM_FEATURES] = {0.0, 0.0, 0.0};
    int orange_count = 0, apple_count = 0;

    for (int i = 0; i < train->size; i++) {
        const FruitSample *s = &ds->data[train->idx[i]];
        if (s->label == FRUIT_LABEL_ORANGE) {
            orange_count++;
            for (int f = 0; f < FRUIT_NUM_FEATURES; f++)
                orange_sum[f] += s->x[f];
        } else {
            apple_count++;
            for (int f = 0; f < FRUIT_NUM_FEATURES; f++)
                apple_sum[f] += s->x[f];
        }
    }
    for (int f = 0; f < FRUIT_NUM_FEATURES; f++) {
        proto_orange[f] = (safeDivide(orange_sum[f],
                          (double)orange_count) >= 0.0) ? +1.0 :
    -1.0;
        proto_apple[f]  = (safeDivide(apple_sum[f],
                          (double)apple_count)  >= 0.0) ? +1.0 :
    -1.0;
    }
}
```

Listing 5: `computeClassPrototypes()` — Equation (1)

8.5. Perceptron Training Loop

```
for (int epoch = 0; epoch < PERCEPTRON_MAX_EPOCHS; epoch++) {
    int updates = 0;
    for (int i = 0; i < train->size; i++) {
        const FruitSample *s = &ds->data[train->idx[i]];
        int y           = toBipolarLabel(s->label);
        int pred_bipolar = (perceptronPredictRaw(&current, s->x)
                            == FRUIT_LABEL_ORANGE) ? +1 : -1;
        if (pred_bipolar != y) {
            for (int f = 0; f < FRUIT_NUM_FEATURES; f++)
                current.w[f] += PERCEPTRON_LR * y * s->x[f];
            current.b += PERCEPTRON_LR * y;
            updates++;
        }
    }
    double valid_acc = evaluatePerceptronAccuracy(&current, ds, valid
    );
    if (valid_acc > best_valid_acc + 1e-12) {
        best_valid_acc = valid_acc;
        best = current;
    }
    if (updates == 0) break;
}
```

Listing 6: The inner epoch loop of `trainPerceptron()` — Algorithm 1

8.6. Stopping Condition: The Zero-Update Epoch

Training terminates when an entire pass over the training set produces zero weight updates (`updates == 0`). This is the classical **Perceptron Convergence Theorem**: on linearly separable data the algorithm is guaranteed to stop in finite epochs [2]. An upper bound of 250 epochs prevents infinite loops on non-separable data.

8.7. Hamming MAXNET Competitive Inhibition

```
for (int iter = 0; iter < MAXNET_ITER; iter++) {
    double next_a[NUM_CLASSES];
    for (int c = 0; c < NUM_CLASSES; c++) {
        double inhibit = 0.0;
        for (int k = 0; k < NUM_CLASSES; k++) {
            if (k != c) inhibit += a[k];
        }
        next_a[c] = a[c] - h->epsilon * inhibit;
        if (next_a[c] < 0.0) next_a[c] = 0.0;   /* ReLU clamp */
    }
    memcpy(a, next_a, sizeof(a));
    int active = 0;
    for (int c = 0; c < NUM_CLASSES; c++) if (a[c] > 0.0) active++;
    if (active <= 1) break;
}
```

```
return (a[0] >= a[1]) ? FRUIT_LABEL_ORANGE : FRUIT_LABEL_APPLE;
```

Listing 7: MAXNET iterative competition inside `hammingPredict()` — Algorithm 4

8.8. Hopfield Hebbian Weight Construction and Synchronous Recall

```
/* ----- Weight construction ----- */
for (int k = 0; k < NUM_CLASSES; k++)
    for (int i = 0; i < FRUIT_NUM_FEATURES; i++)
        for (int j = 0; j < FRUIT_NUM_FEATURES; j++)
            if (i != j)
                hf->W[i][j] += protos[k][i] * protos[k][j];
for (int i = 0; i < FRUIT_NUM_FEATURES; i++)
    for (int j = 0; j < FRUIT_NUM_FEATURES; j++)
        hf->W[i][j] /= FRUIT_NUM_FEATURES;

/* ----- Synchronous recall ----- */
for (int iter = 0; iter < HOPFIELD_MAX_ITER; iter++) {
    double next_state[FRUIT_NUM_FEATURES];
    int changed = 0;
    for (int i = 0; i < FRUIT_NUM_FEATURES; i++) {
        double net = 0.0;
        for (int j = 0; j < FRUIT_NUM_FEATURES; j++)
            net += hf->W[i][j] * state[j];
        next_state[i] = bipolarStep(net);
        if ((int)next_state[i] != (int)state[i]) changed++;
    }
    memcpy(state, next_state, sizeof(state));
    if (!changed) break;
}
```

Listing 8: Hopfield weight construction and recall steps — Algorithms 3 and 5

8.9. Shared Evaluation and Metric Reporting

After collecting predictions from each network, the programme calls:

```
ClassificationReport rpt =
    buildClassificationReportFromArrays(split.truth, predictions,
    split.size);
printClassificationReportWithNames("Perceptron", "Train report",
                                   &rpt, "ORANGE", "APPLE");
```

Listing 9: Metric report call for each network × split combination

This single call prints accuracy, per-class precision/recall/F1, macro and weighted aggregates, and class support — the same eleven metrics used across all other algorithms in the project.

9. Step-by-Step Manual Calculations

Assume the training split yields the following prototypes:

$$\mathbf{p}_{\text{OR}} = [+1, +1, +1], \qquad \mathbf{p}_{\text{AP}} = [+1, -1, -1].$$

All manual calculations use test input $\mathbf{x} = [+1, +1, +1]$ (expected: ORANGE).

9.1. Perceptron: One Corrective Update Trace

Initial state: $\mathbf{w} = [0, 0, 0]$, $b = 0$. Suppose a misclassified APPLE sample $\mathbf{x}' = [+1, -1, -1]$ is encountered ($y^{\text{bip}} = -1$):

$$\begin{aligned}
\text{net} &= 0 \cdot 1 + 0 \cdot (-1) + 0 \cdot (-1) + 0 = 0 \\
\hat{y} &= \text{bipolarStep}(0) = +1 \neq -1 \quad \Rightarrow \text{ update} \\
w_0 &\leftarrow 0 + 0.1 \cdot (-1) \cdot (+1) = -0.10 \\
w_1 &\leftarrow 0 + 0.1 \cdot (-1) \cdot (-1) = +0.10 \\
w_2 &\leftarrow 0 + 0.1 \cdot (-1) \cdot (-1) = +0.10 \\
b &\leftarrow 0 + 0.1 \cdot (-1) = -0.10
\end{aligned}$$

Now predicting the ORANGE input $\mathbf{x} = [+1, +1, +1]$:

$$\text{net} = (-0.10)(1) + (0.10)(1) + (0.10)(1) + (-0.10) = 0.00 \quad \Rightarrow \quad \hat{y} = +1 \Rightarrow \text{ORANGE.} \checkmark$$

9.2. Hamming Network: Full MAXNET Trace

Feedforward layer with $\mathbf{x} = [+1, +1, +1]$:

$$\begin{aligned}
a_{\text{OR}} &= 1.5 + \tfrac{+1}{2}(+1) + \tfrac{+1}{2}(+1) + \tfrac{+1}{2}(+1) = 1.5 + 1.5 = 3.0 \\
a_{\text{AP}} &= 1.5 + \tfrac{+1}{2}(+1) + \tfrac{-1}{2}(+1) + \tfrac{-1}{2}(+1) = 1.5 - 0.5 = 1.0
\end{aligned}$$

MAXNET iteration 1 ($\varepsilon = 1/3$):

$$\begin{aligned}
a'_{\text{OR}} &= \max(0,\ 3.0 - \tfrac{1}{3} \cdot 1.0) = \max(0, 2.667) = 2.667 \\
a'_{\text{AP}} &= \max(0,\ 1.0 - \tfrac{1}{3} \cdot 3.0) = \max(0, 0.000) = 0.000
\end{aligned}$$

Only one unit remains active after iteration 1. $a_{\text{OR}} > a_{\text{AP}} \Rightarrow$ predict ORANGE. ✓

9.3. Hopfield Network: Weight Matrix and Recall Trace

Weight matrix from $\mathbf{p}_{\text{OR}} = [+1, +1, +1]$ and $\mathbf{p}_{\text{AP}} = [+1, -1, -1]$:

$$\begin{aligned}
W_{01} &= \tfrac{1}{3}\big((+1)(+1) + (+1)(-1)\big) = \tfrac{0}{3} = 0 \\
W_{02} &= \tfrac{1}{3}\big((+1)(+1) + (+1)(-1)\big) = 0 \\
W_{12} &= \tfrac{1}{3}\big((+1)(+1) + (-1)(-1)\big) = \tfrac{2}{3} \approx 0.667
\end{aligned}$$

$$W = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0.667 \\ 0 & 0.667 & 0 \end{pmatrix}$$

Recall from $\mathbf{s}^{(0)} = [+1, +1, +1]$:

$$\begin{aligned}
\text{net}_0 &= 0(+1) + 0(+1) = 0 \ \Rightarrow \ s_0^{(1)} = +1 \\
\text{net}_1 &= 0(+1) + 0.667(+1) = 0.667 \ \Rightarrow \ s_1^{(1)} = +1 \\
\text{net}_2 &= 0(+1) + 0.667(+1) = 0.667 \ \Rightarrow \ s_2^{(1)} = +1
\end{aligned}$$

No change: $\mathbf{s}^{(1)} = [+1, +1, +1] = \mathbf{p}_{\text{OR}}$. Exact match ⇒ predict ORANGE, spurious = 0.

Key Insight

All three networks correctly classify $[+1, +1, +1]$ as ORANGE but via fundamentally different mechanisms: the Perceptron uses learned numerical weights; the Hamming network counts prototype agreements; the Hopfield network converges to an energy minimum that matches the stored orange prototype.

10. Expected Programme Output

When executed with a balanced fruit dataset the programme prints four distinct sections. A representative annotated transcript is shown below.

```
  Loaded CSV file:
    Fruit dataset : Data/fruit_dataset.csv
  Dataset summary:
    Total samples : 20
    Train : 12      Validate : 4       Test : 4

============================================================
  PART A -- CLASS PROTOTYPES FROM TRAIN SPLIT
============================================================
  Orange prototype : [+1 +1 +1]
  Apple prototype  : [+1 -1 -1]

============================================================
  PART B -- PERCEPTRON NETWORK
============================================================
  Learning rate : 0.10
  Max epochs    : 250
  Final weights : [0.1000, 0.2000, 0.3000]  bias = -0.1000

  Perceptron | Train report
    Accuracy  : 91.67 %
    ORANGE -- Precision: 1.00  Recall: 0.86  F1: 0.92  Support: 7
    APPLE  -- Precision: 0.83  Recall: 1.00  F1: 0.91  Support: 5

============================================================
  PART C -- HAMMING NETWORK
============================================================
  Feedforward weights (prototype / 2):
    Orange row : [+0.50 +0.50 +0.50]
    Apple row  : [+0.50 -0.50 -0.50]
  ...

============================================================
  PART D -- HOPFIELD NETWORK
============================================================
  Weight matrix W:
    [ +0.0000  +0.0000  +0.0000 ]
    [ +0.0000  +0.0000  +0.6667 ]
    [ +0.0000  +0.6667  +0.0000 ]
  Spurious recalls on train split      : 0
  Spurious recalls on validation split : 0
  Spurious recalls on test split       : 0
```

Listing 10: Representative console output of `neural_net_classifier.c`

11. Compilation and Execution in C

11.1. Prerequisites

- A C99-compatible compiler (GCC $\geq$ 4.8, Clang $\geq$ 3.5, or MSVC with C11).
- GNU `make` (or equivalent).
- `Data/fruit_dataset.csv` present in the project root.

11.2. Build with the Shared Makefile

```
make
```

Listing 11: Compile all targets from the project root

This links `neural_net_classifier.c` against `ml_common.c` and the math library (`-lm`), producing the `neural_net_classifier` executable.

11.3. Run with Default Dataset Path

```
./neural_net_classifier
```

Listing 12: Execute with the default CSV path (`Data/fruit_dataset.csv`)

11.4. Run with an Explicit CSV Path

```
./neural_net_classifier Data/fruit_dataset.csv
```

Listing 13: Pass a custom dataset path as the first argument

11.5. Interactive Prediction Mode

After all reports are printed the programme enters an interactive loop. The user selects a network (1 = Perceptron, 2 = Hamming, 3 = Hopfield), then enters bipolar values for shape, texture, and weight. The programme responds with the predicted label and, for the Hopfield network, a spurious recall indicator. Enter `q` at any prompt to exit.

12. Results and Discussion

12.1. Perceptron

The Perceptron is the only network in the programme that actually modifies numerical weights from labelled training examples. On linearly separable fruit data its error-corrective updates converge quickly (often within 10–30 epochs on a balanced 20-sample dataset). The best-validation checkpoint prevents overfitting to the training split. Printed weights reveal the relative importance of each feature: a large positive w_2 (weight feature) signals that heaviness is a strong predictor of ORANGE.

12.2. Hamming Network

The Hamming network never iterates over examples — it is fully defined once the prototypes are computed. It achieves perfect accuracy on any test sample that shares the majority feature profile of a training class. However, because it stores only one prototype per class, it is fragile when class overlap is high or when the training prototypes are noisy.

12.3. Hopfield Network

The Hopfield network stores patterns as energy minima of a Lyapunov function:

$$E = -\frac{1}{2}\sum_{i \neq j} W_{ij} s_i s_j. \tag{9}$$

Synchronous recall is guaranteed to decrease E monotonically. With only $P = 2$ stored patterns and $n = 3$ neurons, the network operates well within the theoretical storage capacity of $\approx 0.138n \approx 0.414$ patterns [3], so spurious states should not appear on well-formed inputs. The programme reports spurious recall counts for transparency.

12.4. Comparative Metric Summary

Metric (Test split)	Perceptron	Hamming	Hopfield
Training style	Supervised, iterative	Prototype, one-shot	Hebbian, one-shot
Memory stored	Weight vector + bias	Two prototype rows	3×3 matrix
Spurious states	N/A	N/A	Reported per split
Hyperparameters	LR, epochs	ε	Max iterations
Recall type	Discriminative	Nearest prototype	Associative

13. Advantages of the Three-Network Approach

A1. Theoretical breadth. Three distinct learning paradigms (error-corrective, prototype-matching, associative memory) are explored with one shared codebase and one common evaluation layer.

A2. Zero external dependencies. The entire programme compiles with `gcc -lm`; no external libraries are needed.

A3. Bipolar encoding efficiency. The $\{-1, +1\}$ representation allows the Hopfield outer-product rule and the Hamming similarity computation to be exact integer operations in fixed-feature space.

A4. Deterministic reproducibility. A fixed shuffle seed (`FRUIT_SPLIT_SEED = 815`) ensures that every run on the same dataset produces identical train / validate / test splits.

A5. Unified evaluation. All three networks report identical metric sets through `ml_common`, making numerical comparison straightforward.

A6. Interactive CLI. The interactive prediction mode lets users probe all three networks in real time without recompiling or scripting.

14. Limitations of the Current Implementation

L1. Binary classification only. All three networks handle exactly two classes. Extension to $C > 2$ would require one-vs-rest wrappers or architectural changes.

L2. Perceptron: no kernel extension. The linear decision boundary cannot model curved separating surfaces. Non-linearly separable data may never converge within 250 epochs.

L3. Hopfield: limited storage capacity. The classical Hopfield network can reliably store only $\approx 0.138 \cdot n$ patterns without spurious states [3]. For $n = 3$, fewer than one additional pattern can be safely added.

L4. Hamming: prototype rigidity. Only one prototype per class is retained; intra-class variability is completely discarded.

L5. Fixed bipolar encoding. The dataset requires features pre-encoded as $\{-1, +1\}$. Continuous features would need a discretisation or normalisation step not currently provided in this file.

L6. Synchronous Hopfield update. The synchronous update rule used here can, in theory, oscillate between two states that are not fixed points. Asynchronous updates provide stronger convergence guarantees [3].

15. Appendix: Ready-to-Use Data Files

15.1. `Data/fruit_dataset.csv` — Training and Evaluation File

The file below provides a complete, self-contained 20-sample fruit dataset balanced between 10 ORANGE and 10 APPLE examples. The programme splits this internally at runtime (60/20/20) using seed 815.

```
shape,texture,weight,label
+1,+1,+1,ORANGE
+1,+1,+1,ORANGE
+1,+1,+1,ORANGE
-1,+1,+1,ORANGE
+1,-1,+1,ORANGE
+1,+1,+1,ORANGE
+1,+1,-1,ORANGE
+1,+1,+1,ORANGE
-1,+1,+1,ORANGE
+1,+1,+1,ORANGE
+1,-1,-1,APPLE
+1,+1,-1,APPLE
-1,-1,-1,APPLE
+1,-1,-1,APPLE
-1,-1,-1,APPLE
+1,-1,-1,APPLE
+1,+1,-1,APPLE
-1,-1,+1,APPLE
+1,-1,-1,APPLE
-1,-1,-1,APPLE
```

Listing 14: `Data/fruit_dataset.csv` — 20 labelled fruit samples

15.2. Sample Interactive Test Vectors

The following vectors can be entered during the interactive CLI session to manually verify each network's prediction logic:

shape	texture	weight	Expected	Rationale
+1	+1	+1	ORANGE	Classic orange prototype
+1	−1	−1	APPLE	Classic apple prototype
−1	−1	−1	APPLE	Elongated, rough, light
+1	+1	−1	APPLE	Round smooth but light
−1	+1	+1	ORANGE	Heavy overrides other features

16. Conclusion and Future Directions

16.1. Conclusion

This report presented a complete, single-file C99 implementation of three foundational neural network classifiers — Perceptron, Hamming Network, and Hopfield Network — applied to a bipolar fruit-recognition task. All three architectures were derived from first principles, implemented without external libraries, and evaluated under a unified metric framework.

The Perceptron demonstrated that a simple weight-update rule on labelled examples can reliably learn a linear decision boundary, with the added robustness of validation-based checkpointing. The Hamming Network showed that a two-layer competitive architecture can reduce classification to a prototype-distance problem solved in a single feedforward pass plus a lightweight MAXNET competition. The Hopfield Network illustrated associative memory: patterns are embedded as energy minima in a fully connected recurrent system and recovered by energy-descending iteration, with the programme transparently flagging any spurious convergences.

Together, the three networks cover the essential design space of early neural computation — supervised discriminative learning, unsupervised prototype matching, and recurrent associative recall — in fewer than 720 lines of readable, well-structured C.

16.2. Future Directions

F1. Multi-layer Perceptron. Adding a hidden layer and backpropagation would lift the linear-separability constraint.

F2. Pocket Algorithm. Replacing the standard Perceptron rule with the pocket algorithm would guarantee the best separator found on non-separable data [5].

F3. Extended Hamming Network. Storing multiple prototypes per class (one per cluster) would capture intra-class variability.

F4. Modern Hopfield Networks. The 2016 dense associative memory model [6] exponentially increases storage capacity and directly generalises to the attention mechanism in transformers.

F5. Continuous Feature Support. Adding a z-score normalisation step in the loader would allow continuous biometric features (e.g. circumference in mm) to be used alongside the existing bipolar architecture.

F6. Asynchronous Hopfield Update. Replacing synchronous with asynchronous updates would eliminate the risk of 2-cycles in recall and provide stronger convergence guarantees.

References

[1] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.

[2] F. Rosenblatt, "The Perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

[3] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proceedings of the National Academy of Sciences*, vol. 79, no. 8, pp. 2554–2558, Apr. 1982.

[4] R. P. Lippmann, "An introduction to computing with neural nets," *IEEE ASSP Magazine*, vol. 4, no. 2, pp. 4–22, Apr. 1987.

[5] S. I. Gallant, "Perceptron-based learning algorithms," *IEEE Transactions on Neural Networks*, vol. 1, no. 2, pp. 179–191, Jun. 1990.

[6] H. Ramsauer *et al.*, "Hopfield networks is all you need," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[7] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson Education, Upper Saddle River, NJ, 2009.

[8] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, New York, 2006.

[9] J. Hertz, A. Krogh, and R. G. Palmer, *Introduction to the Theory of Neural Computation*. Addison-Wesley, Reading, MA, 1991.

[10] J. M. Zurada, *Introduction to Artificial Neural Systems*. West Publishing, St. Paul, MN, 1992.

[11] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

[12] J. Bruck and V. P. Roychowdhury, "On the number of spurious memories in the Hopfield model," *IEEE Transactions on Information Theory*, vol. 36, no. 2, pp. 393–397, Mar. 1990.